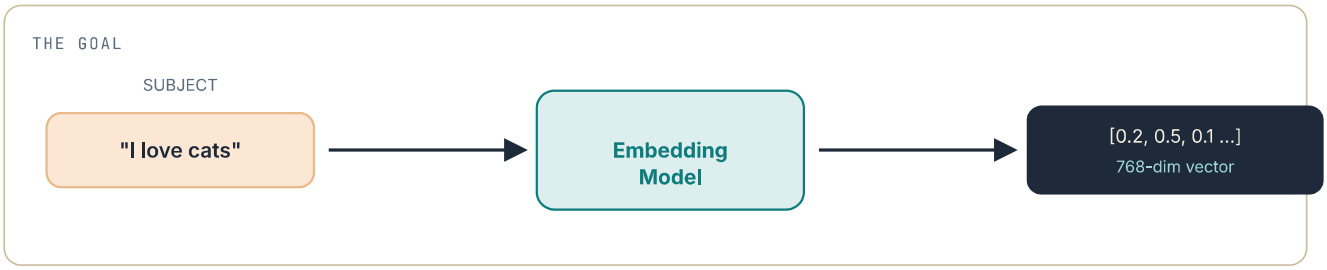
EMBEDDING MODELS · PART 1 OF 4

How an Embedding Model Gets Trained

From a plain transformer to a vector that actually means something — architecture choice, the per-token problem, and pooling.



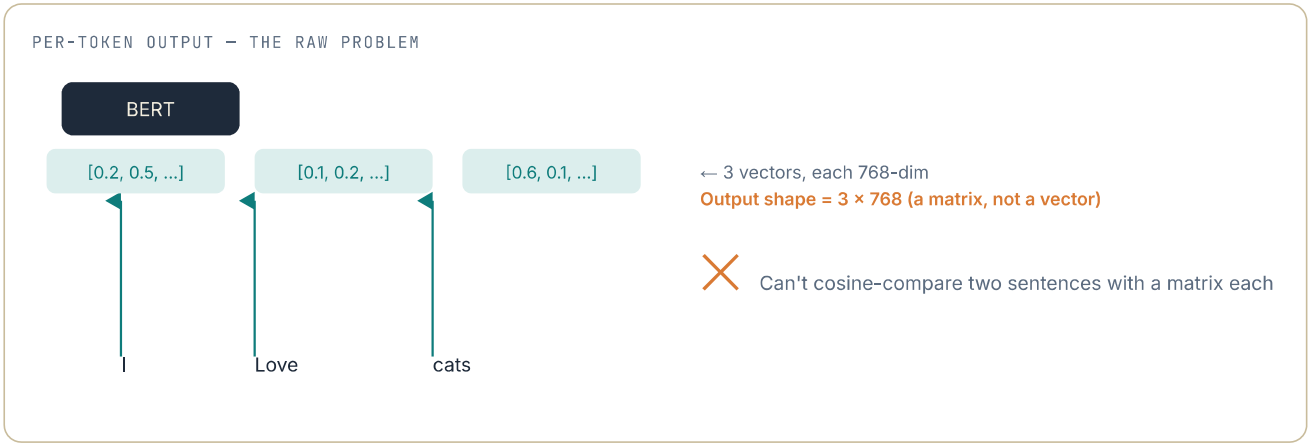
An embedding model is a **function** that maps a subject into a vector representation which captures its **semantic meaning** — keeping similar-meaning vectors close together, and pushing dissimilar ones far apart.

STEP 1 Choose a base architecture

Take a **transformer encoder** — only the encoder part, like **BERT**. It already understands language structure through self-attention; we just need to shape its output into something usable.

PROBLEM — TRANSFORMER OUTPUT

A transformer gives one vector **per token**, not one vector for the whole sentence. "I love cats" → 3 separate 768-dim vectors. We need exactly one.



SOLUTION Pooling layer — Mean Pooling

Pooling means "combine." Mean pooling takes the average of all token vectors — giving every word equal say in the final meaning.

```
sentence_vector = ( vector(I) + vector(love) + vector(cats) ) / 3
```

RESULT

One clean 768-dim sentence vector — poori sentence ka balanced summary, using info from every token.

ALTERNATIVE

Use the **[CLS]** token's own vector instead of averaging — a special token designed exactly for this. Covered next →

encoder per-token output mean pooling embedding-model-notes

01 / 04

EMBEDDING MODELS · PART 2 OF 4

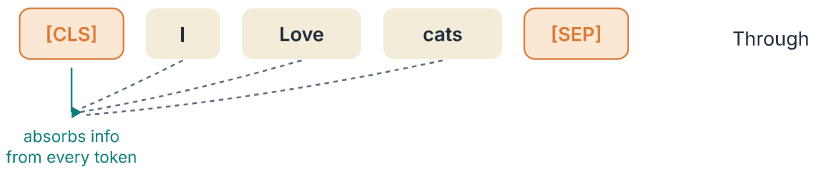
The [CLS] Token, and Which Pooling Wins Where

The second way to collapse per-token vectors into one — and why bi-encoders and cross-encoders make opposite choices.

SOLUTION 2 [CLS] Token

[CLS] = "classifier token." BERT inserts it before every sentence, and **[SEP]** after — both are artificial markers, not real words.

SPECIAL TOKENS AROUND THE SENTENCE



self-attention, [CLS] attends to every other token in every layer — by the final layer, its vector is effectively a learned **summary** of the whole sentence. sentence_vector = vector([CLS]) ← no averaging needed

WHY IT WORKS

During BERT's pre-training (Masked Language Modeling + Next Sentence Prediction), the model is forced to concentrate whole-sentence meaning into [CLS] — so after training, that one vector is already a trained summary.

SHOWDOWN Mean Pooling vs [CLS] — which is better?

Aspect	Mean Pooling	[CLS] Token
What it does	Averages every token vector	Takes one special token's vector
Info used	All tokens, equal weight	Whatever [CLS] absorbed via attention
Out-of-the-box quality	Consistently better (SBERT, 2019)	Weak unless fine-tuned for it — raw BERT's [CLS] was mainly trained for Next-Sentence-Prediction, a weak signal
Used by	Sentence-BERT, E5, BGE, INSTRUCTOR, OpenAI embeddings	Original BERT classification head

VERDICT — BI-ENCODERS

Mean pooling wins for standalone sentence embeddings — because the vector must be reusable and generic, and averaging spreads that responsibility across every token instead of one bottleneck.

EXCEPTION Cross-encoders flip the choice

Bi-encoder: query and doc go through the model *separately* — [CLS] must become a generic, reusable summary with zero knowledge of what it'll be compared against. Harder job → lossier.

Cross-encoder: query + doc go in *together* as one input — `[CLS] query [SEP] doc [SEP]`. Every token can attend to every other token directly, so [CLS] only has one narrow job: output **this specific pair's** relevance score.

WHY CROSS-ENCODER'S [CLS] IS LESS LOSSY

It's task-specific, not general-purpose — full query↔doc interaction already happened in the attention layers before [CLS] condenses it into one relevance score via a dedicated classifier head.

CLS token mean vs CLS cross-encoder embedding-model-notes

02 / 04

EMBEDDING MODELS · PART 3 OF 4

Teaching the Model: Contrastive Training

A random encoder + pooling layer is meaningless. Here's how it learns to pull similar things close and push dissimilar things apart.

STEP 2 Create pairs

We need a function that trains the model on the **contrast** between similar and dissimilar text. Start by building **(anchor, positive)** pairs.

PROBLEM — TRIVIAL COLLAPSE

If loss only says "minimize distance(anchor, positive)", the model finds a cheat: map **everything** to the same vector. Distance becomes 0, loss is "solved" — but the embedding is useless (cat = car = anything).

```
"I am XYZ" → [1.0, 1.1, 5.2, 3.0]   ≈   "I love ABC" → [1.0, 1.1, 5.2, 3.0]
```

FIX Add negatives — Triplet Loss

```
Loss = max( 0,   d(anchor, positive) - d(anchor, negative) + margin )
```

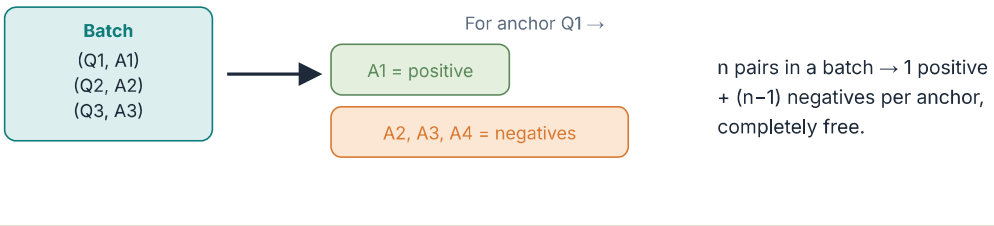
If the negative is already farther than positive + margin → loss = 0, nothing to learn. Otherwise the loss pushes positive closer, negative farther. Margin is a fixed hyperparameter — like a safety buffer, doesn't change during training.

LIMITATION OF TRIPLET LOSS

Only sees **one negative at a time** → weak signal, slow learning.

SCALE In-batch negatives

1000 (query, correct-answer) pairs available. Pick a batch of **n = 4**. For anchor Q1, its own A1 is positive — the other three answers (A2, A3, A4) automatically become negatives. Free negatives, zero extra labeling.



UPGRADE InfoNCE (Multiple Negatives Ranking Loss)

$$\text{Loss} = -\log\left(\exp(\text{sim}(a, \text{pos})) \,/\, [\, \exp(\text{sim}(a,\text{pos})) + \exp(\text{sim}(a,\text{neg}_1)) + \exp(\text{sim}(a,\text{neg}_2)) + \ldots \,] \right)$$

This is a **softmax** — `exp()` turns similarity into a positive score and amplifies gaps (exp(0.9)=2.46 vs exp(0.1)=1.10, a 2.24× spread from a 0.8 difference). All negatives are compared to the anchor *at once*, so the model gets a much richer signal per

step than triplet loss.

ROBUST VERSION — ADDS TEMPERATURE (T)

Loss = −log(exp(sim/τ) / Σ exp(sim_i/τ)). τ sharpens or softens the similarity gap. Used in Sentence-BERT, BGE, E5. τ is usually fixed, but some models (e.g. CLIP) make it learnable too.

---

trivial collapse triplet loss in-batch negatives InfoNCE embedding-model-notes

EMBEDDING MODELS · PART 4 OF 4

Matryoshka: One Model, Many Dimensions

A trained 1024-dim embedding is expensive to search. Matryoshka Representation Learning lets you truncate it — on purpose, and for free.

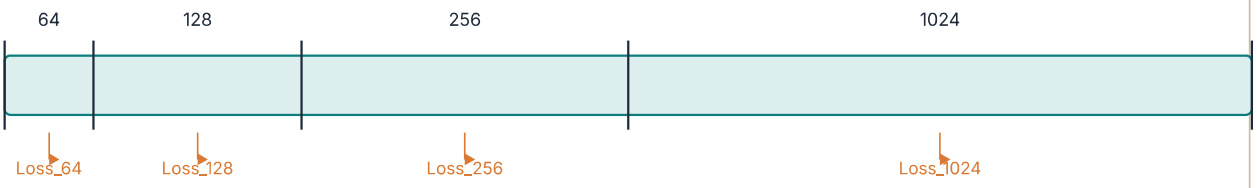
PROBLEM

A full-size vector (say 1024-dim) is slow to search and expensive to store at scale. Training a separate smaller model is wasteful — new training run, new model to maintain.

IDEA Nested loss on prefix slices

Instead of computing InfoNCE only on the full 1024-dim vector, slice it into nested prefixes — **64, 128, 256, 1024** — and compute InfoNCE loss *on each slice separately*, then sum them.

NESTED SLICES OF THE SAME VECTOR



One forward pass builds the full 1024-dim vector — slicing is just indexing, no extra compute

```
Loss_total = Loss_64 + Loss_128 + Loss_256 + ... + Loss_1024   →   one .backward()
```

WHY IT WORKS Gradient frequency, not a hardcoded rule

Nobody tells the model "put important stuff first." It emerges — because early dimensions appear in *every* slice's loss, so they get gradient updates far more often:

Dimension range	Appears in losses	Gradient updates per step
0 – 64	Loss_64, Loss_128, Loss_256, Loss_1024	**4× — maximum signal**
65 – 128	Loss_128, Loss_256, Loss_1024	3×
129 – 256	Loss_256, Loss_1024	2×
257 – 1024	Loss_1024 only	1× — least pressure

EMERGENT BEHAVIOUR

Because dim 0–64 is under pressure from four losses at once, the model is forced to pack the most important semantic signal there — front slices become information-dense "by necessity," not by instruction.

PAYOFF Truncate at inference, no retraining

BEFORE MATRYOSHKA

Want a smaller vector? Train a whole new model at that dimension.

AFTER MATRYOSHKA

Just slice: `embedding[:256]`, normalize, done. Front dims already carry the meaning — degradation is graceful, not random.

One-liner for interviews: *"Matryoshka trains with nested InfoNCE losses on prefix slices of the same vector, so early dimensions get more gradient signal and naturally become information-dense — enabling runtime truncation without retraining."*

Matryoshka nested loss truncation embedding-model-notes

04 / 04